

Pipelining

Pipelining এর main concern হচ্ছে Execution time improve করা।

যেমন মেশিন কোড program এর Resource use করে মেশিনে যাওয়া resource এর free করে দিতে সাহায্য করে।

Two cycle of pipeline:

i) Bus interface unit (BIU)

এই main কাজ হচ্ছে RAM থেকে বা RAM এর code segment থেকে Instruction fetch (IF) করা।

এই portion এ আছে

↓
Add: AX, BX
Jump:

- segment Register (CS, DS, SS, ES)
- Offset " (IP)
- instruction queue

→ FIFO

→ First 4 byte নিয়ে একটি instruction এর সমান।

instruction জুনির fetch করে
এই queue এ রাখা হয়।

BIU চি করে RAM থেকে যা 20 bit AB নিয়ে কাজ করে।

ii) Execution Unit (EU):

এই cycle চি mainly fetch করে instruction queue থেকে নিয়ে execute করে।

Two way of execution:

i) Instruction Decode (ID)

আমরা instruction queue থেকে আমরা যা নিয়ে decode করে নেই সে চি চি internal হয়ে আছে যা চি perform করে লাগবে।

ii) Instruction execution (IE)

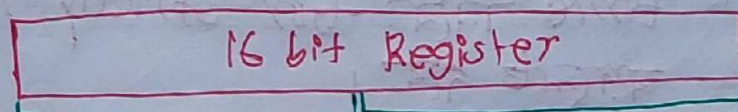
mainly আমরা instruction চি করে এর operation সুন্দর perform করে চি।

~~এই~~ portion

EU portion চি করে

- ALU, general purpose register (AX, BX, CX, ...)
- Temporary register
- Offset Register except IP (SP, BP, SI, DI)

N.B



8 bit: AH

8 bit: AL

Three case to complete the pipelining:

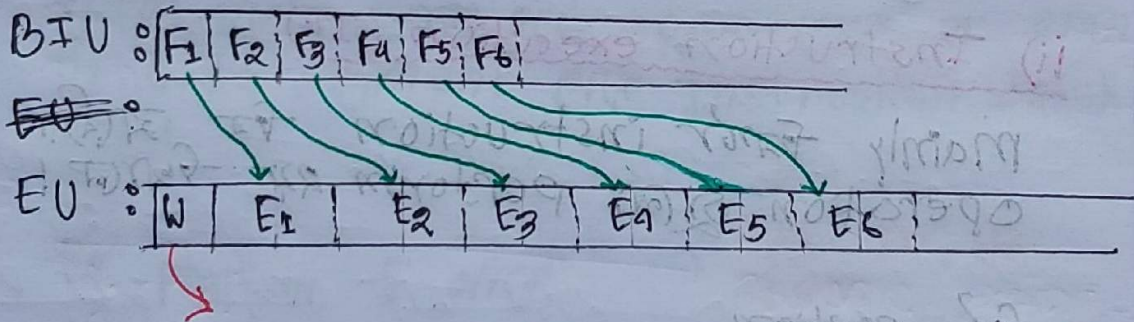
Case:1

এর ক্ষেত্রে just normal case, sequentially fetch করে and sequentially execute করে।

Ex:

1. Add:	AX, BX
2. Sub:	CX, BX
3. Add:	BX, CX
4. XOR:	AX, BX
5. XOR:	BX, CX
6. mul:	CX, BX

Assume first instruction
~~fetch করে~~
execute করে করে রত
instruction fetch করে
লাগে।



প্রথমবার মধ্য fetch করে চিহ্নিত করে কিছু নাহ
so EU চিহ্ন idle থাকবে ~~এ~~ জানি Wait থাকবে।

N.B mainly BIU ও EU এর ~~এ~~ আবেদ
time নির চিহ্ন strictly maintain না
করলে চলবে।

কিন্তু EU এর execution এর same
sequence হবে সবসময়।

Case:2 mov

MOV instruction execute করার সময় Decode করে দেখে যে memory থেকে Data লাগবে so EU, BIU কে ~~কিছু~~ ^{জানাবে} যে সময় সময় memory থেকে Data Read ~~করে~~ ^{হবে} নিজে, তখন EU must wait করবে।

N.B EU সময় (মতো) wait করা শুরু করবে যিহা তখনই BIU এর কাজ শেষ Data Read করে নিজে।

BIU:

1	2	3	4	5	6	7	8	9	10
				Read Data					

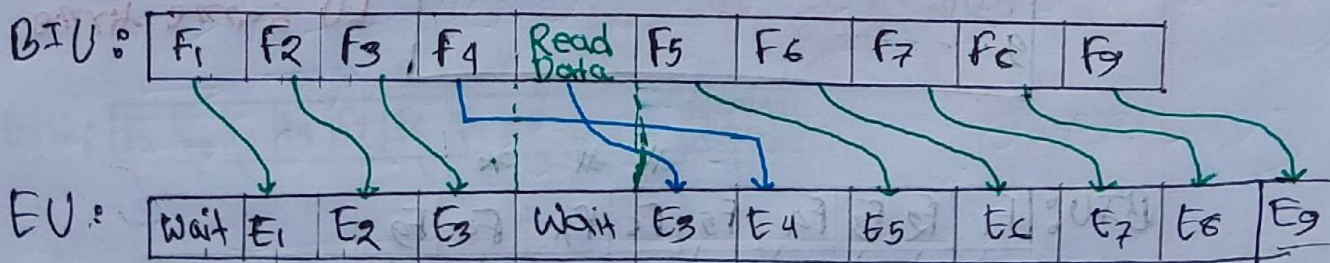
EU:

1	2	3	4	5	6	7	8	9	10
				Wait					

Ex:

1. Add
2. ...
3. move DX, [2AFB2H]
- ...
9. ...

BIU and EU same time লাগে (Assume)



যিহা সময় EU F₃ কে execution শুরু করে তখন Decode করছে মোব আছে যে সময় EU wait করেছিলো যে BIU Data Read করেছিলো।

Case: 3 Jump:

Jump starts current line (which is the line to move to) and instruction execute. But, after the jump instruction, the processor must wait before fetching the next instruction.

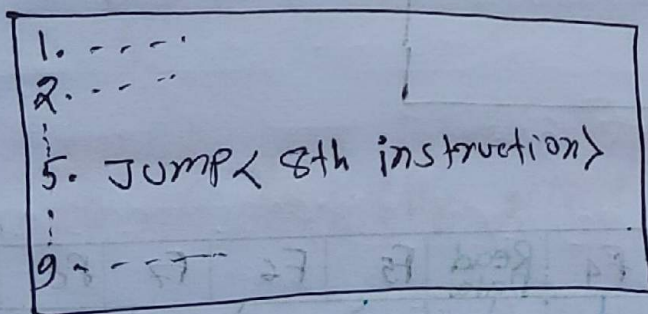
When jump is wait, the processor must wait before instruction fetch starts.

N.B Jump instruction must fetch skip and processor # sign must identify instruction.

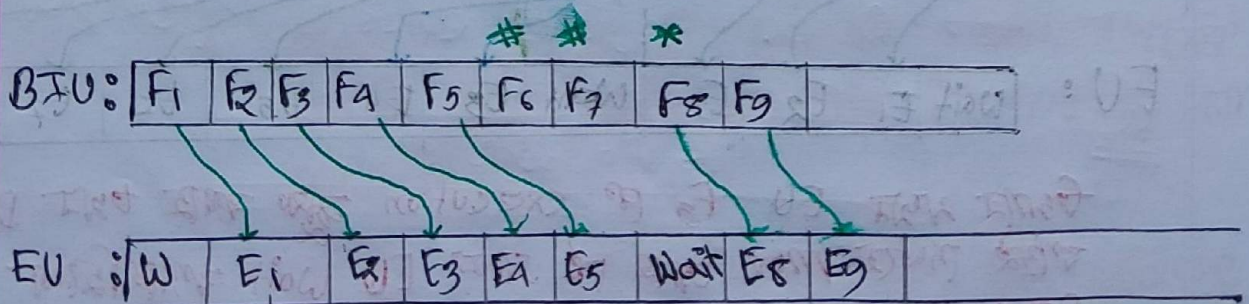
Jump instruction must instruction/fetch and skip and processor * sign must denote instruction.

Jump instruction queue is not the same as discard and jumping address (which is the address to fetch).

Ex:

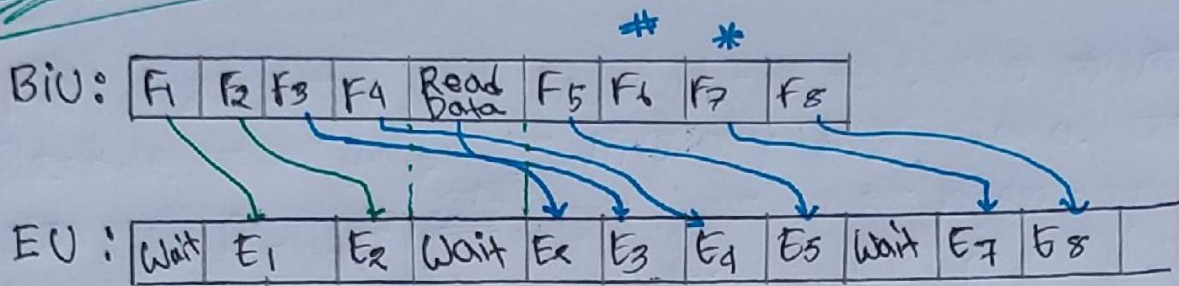


Assume processor execute instruction 5th instruction and skip instruction 6th instruction. But, processor must wait before fetching instruction 8th instruction.



: Discarded
* : Jump destination

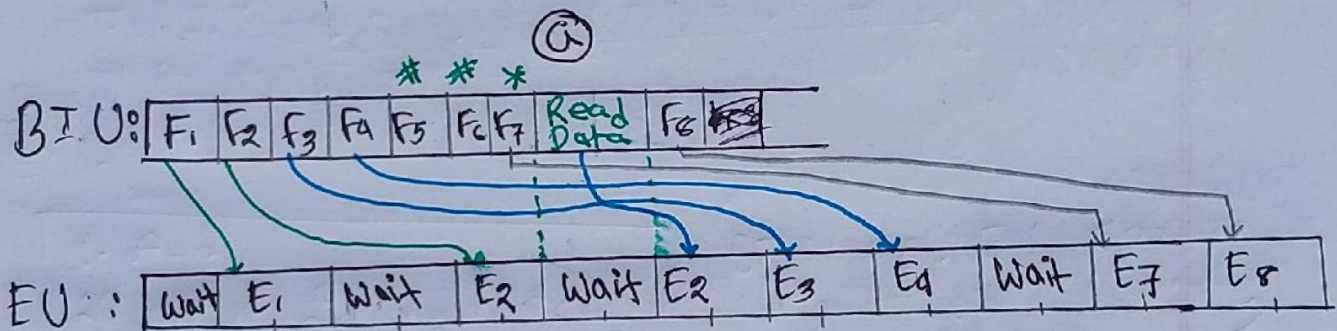
Ex slide 15



#; Discarded

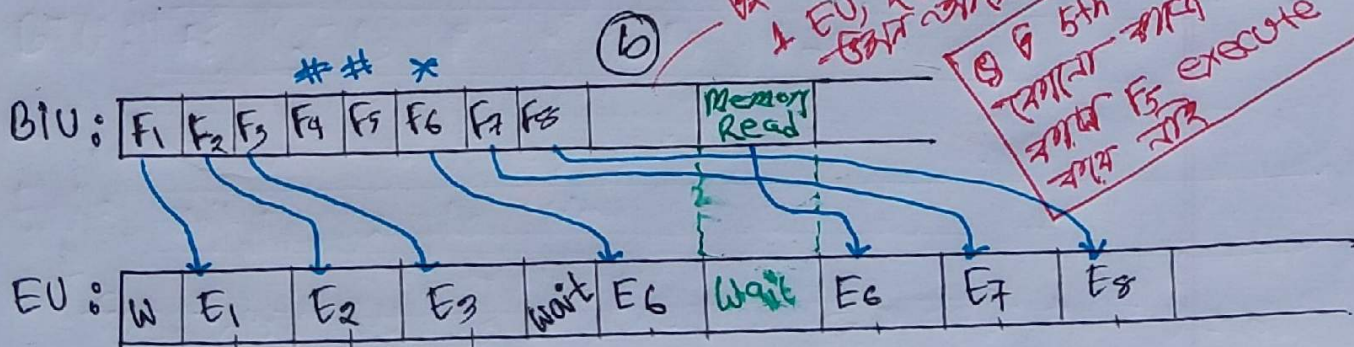
*; Jump destination

Fall 25 Q2(a)



#; Discarded

*; Jump destination

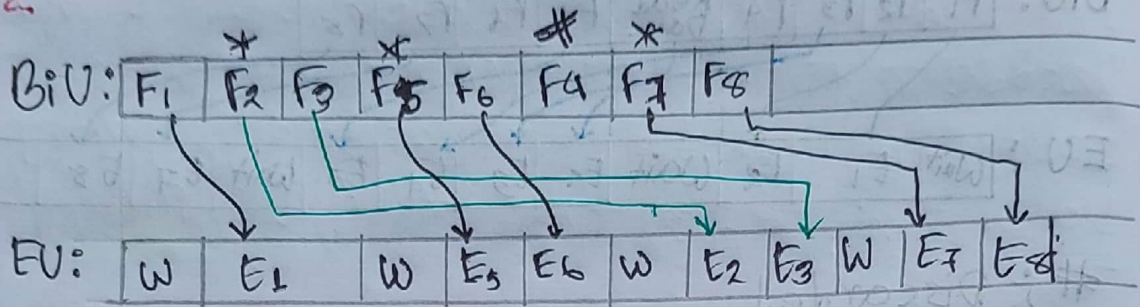


#; Discarded

*; Jump destination

Q2 (a) & (b) idly state mein hai
1 EU, 2 BIU hi use ho rahi hai
Q2 (b) mein jump hai
F5 execute ho rahi hai

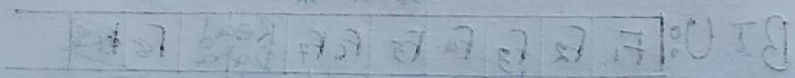
summer: 25 93(a)



#: discarded

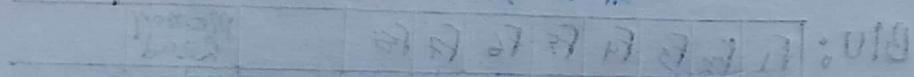
*: jump destination

①



#: discarded
*: jump destination

②



#: discarded
*: jump destination